- Roborace 2024 - Lösung - MPG Happy Hippos
  - Funktionen
    - Zentrale Funktionen
    - Weitere Funktionen
  - Code Erklärung
    - Strategie
    - Entlang von Wänden und Ecken fahren
    - Andere Häuser finden
    - Drehen
  - Aufgabenstellung
    - Die Aufgabenstellung
- Literatur
- Danksagungen
  - Betriebssystem und Bibliothek
  - Dokumentation
  - Simulation
  - Namensgebungskonventionen

Roborace 2024 - Lösung - MPG Happy Hippos

Eine mögliche Lösung zur Aufgabe des diesjährigen Roborace Stuttgart mithilfe der ev3dev2 Python Bibliothek.



Funktionen

Kurz gesagt, der Roboter ist als Lieferroboter programmiert.

Detaillierte Erklärungen zur Aufgabe können im Anhang "Aufgabenstellung" und auf der Homepage der Universität Stuttgart gefunden werden.

Zentrale Funktionen

- entlang von Wänden und Ecken fahren (mithilfe eines PID-Reglers [2])
- andere Häuser finden
- zu anderen Häusern fahren (optimaler Kurs wird mithilfe von Trigonometrie berechnet)

Weitere Funktionen

- Töne abspielen
- unterschiedliche Gradzahl drehen (sowohl relativ zur aktuellen als auch zur ursprünglichen Position)
- Pakete abwerfen

Code Erklärung

Die Kontrolle des Roboters erfolgt mittels mehrerer ineinander verschachtelter Schleifen und paralleler Threads.

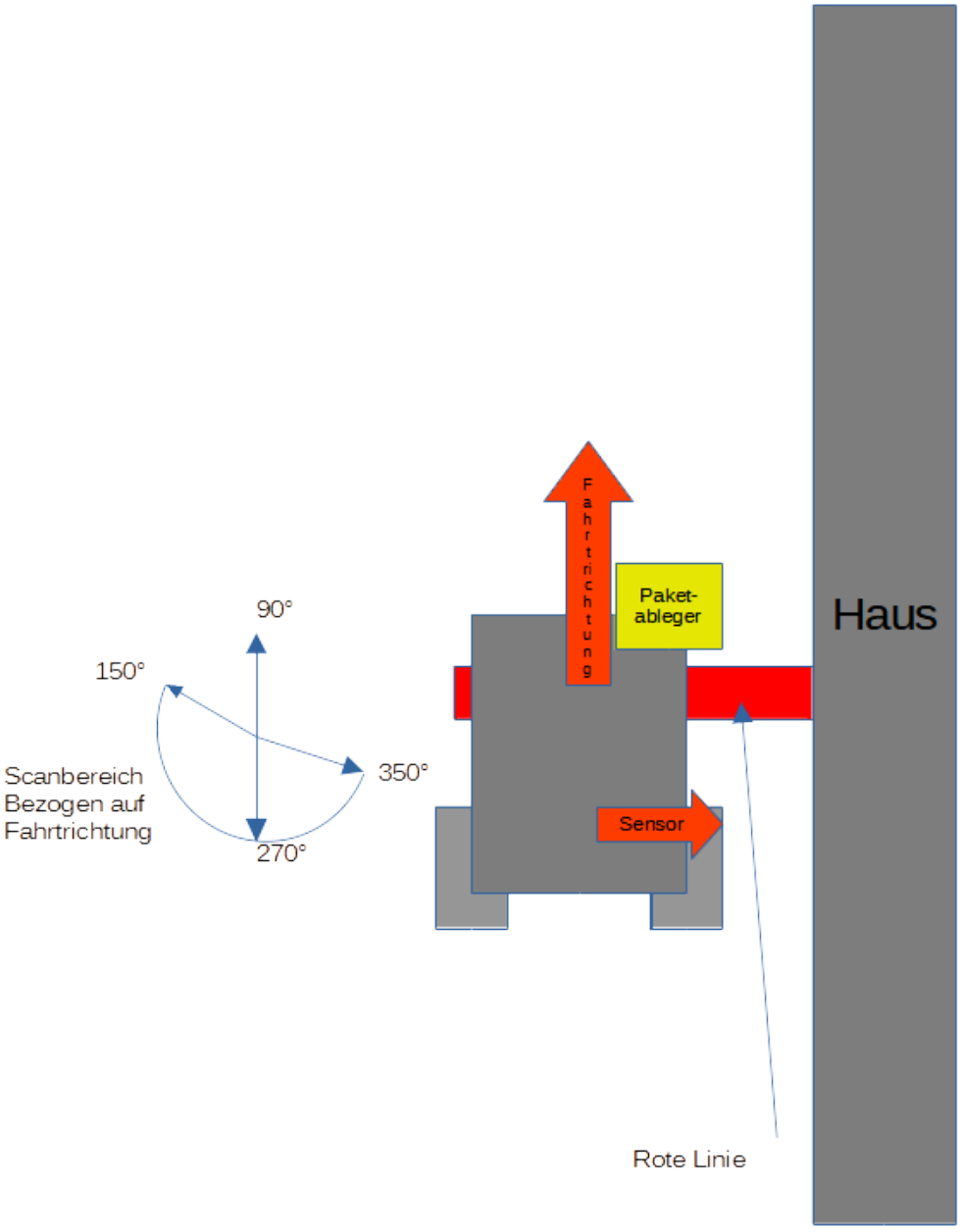
In den Threads werden die Sensorwerte des Ultraschallsensors zur Abstandserfassung und des Gyrosensors zur Winkelerfassung ausgelesen, aufbereitet und den Routinen des Hauptprogramms (main) mittels Klassen zur Verfügung gestellt.

Der Farbsensor wird direkt aus dem Hauptprogramm angesprochen.

Aus Gründen der Vereinfachung wurde auf eine zusätzliche Mittelschicht (middleware), die eine Abarbeitung in festen Zeitrastern und Bibliotheksunabhängigkeit zur Hardware ansteuerung ermöglichte, verzichtet. Gleiches gilt für den Verzicht auf ein Hardware spezifisches Betriebssystem (OS, Robo-OS).

Strategie

Der Roboter fährt an der Wand eines Hauses entlang, bis der Farbsensor eine rote Linie erkennt. Während der Fahrt versucht der Roboter mithilfe des Abstandsensors einen Abstand von 13cm zur Wand zu halten. Sobald die Rote Linie detektiert wurde hält der Roboter an. Es folgt eine Drehung, die einen Winkelbereich von ~180° abdeckt.

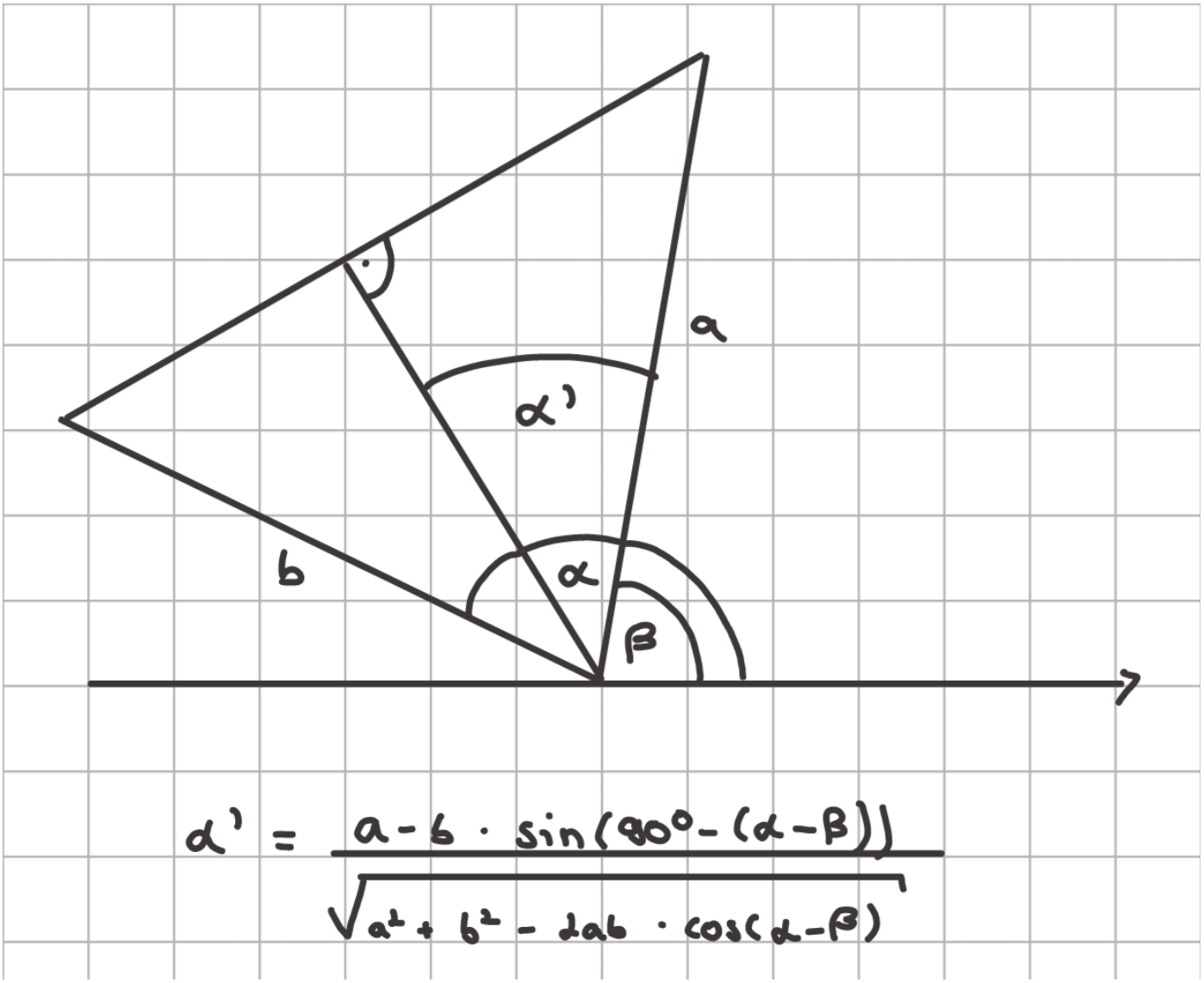


Während der Drehung wird kontinuierlich der Abstand erfasst und ausgewertet. Über eine Kantenerkennung werden die Positionen der Ecken der in der Nähe stehenden Häuser erfasst und deren Abstände und Winkel in einem Dictionary gespeichert.

Die Wand mit dem kürzesten Abstand soll im rechten Winkel angefahren werden. Dieser Ansteuerwinkel wird mittels Cosinussatz errechnet.

```
anOfAprch = ((((a - b * math.sin(math.radians(90-(beta-alpha)))) / math.sqrt(((a*a)
+ (b*b)) - (2*a*b * math.cos(math.radians((beta-alpha)))))) * 180 / math.pi) +
alpha) - 90
*detect_houses_advanced.py line:183*
```

detect__houses_advanced.py, Line: 183



beta ist der Winkel des Sensors bei der ersten Ecke relativ zur Hauswand, alpha ist der Winkel des Sensors bei der zweiten Ecke relativ zur Hauswand, a ist die Distanz bis zur ersten Ecke, b die Distanz bis zur zweiten Ecke, alpha' ist der Winkel, den man über beta hinausgehen muss, um auf der Senkrechten der Hauswand zu sein. Bevor die nächste Hauswand angefahren wird, wird jedoch noch das Paket abgeworfen. Damit der Roboter nicht über das abgeworfene Paket fährt, dreht er sich kurz in Richtung Hauswand, läßt das Paket fallen und steuert dann die nächste Wand an.

Da der Sensor um 90° zur Fahrtrichtung gedreht ist, kann er während des Anfahrens der Wand keine Distanzdaten zu dieser liefern. Darum wird, ebenfalls mit Trigonometrie, die Distanz zur nächsten Wand berechnet.

```
distToNextHouse = abs(math.cos(anOfAprch - alpha)) * a
```

detect__houses_advanced.py, Line: 193

Von dieser berechneten Distanz wird noch der Sollabstand abgezogen, damit der Roboter nicht mit der Wand kollidiert (da eine Kollision mit einem Haus zu einer Zeitstrafe führt).

Entlang von Wänden und Ecken fahren

Folgendes ist die Grundlage für das Entlangfahren an Wänden. Das Programm nutzt die Differenz zwischen der aktuellen Distanz zu einem Haus und der Distanz, die der Roboter haben soll, und setzt diese in eine PID-Regler-Formel ein. Der Ausgabewert "turnratio" wird dann zur Steuerung genutzt.

```
error = ultrasonic_sensor.distance_centimeters - goal
integral += (error * deltaTime)
derivative = (error - previous_error) / deltaTime

turnratio = (kp * error) + (ki * integral) + (kd * derivative)

steering_drive.on(turnratio, 20)
```

steering_drive.on(turnratio, 20) steuert den Motor mit dem Korrekturwert an. Die Ermittlung der Differenz zwischen Solldistanz und aktueller Distanz geschieht in der "main" Function des Mouduls parallel_wall_drive.py.

Andere Häuser finden

Folgendes ist die Basis für das Erkennen von Hausecken. Das Programm berechnet permanent die Differenz zwischen der aktuellen und der letzten Distanz und schaut nach großen Sprüngen. Wenn ein Wertesprung groß genug ist, wird eine Hausecke erkannt, und der aktuelle Winkel sowie die Distanz zum nächsten Haus wird gespeichert.

Momentan gibt es bei scharfen Ecken das Problem, dass der Roboter entweder viel zu stark oder viel zu wenig dreht (je nach p, i und d). Um dies zu lösen, müsste man dem Algorithmus die Fähigkeit geben, Ecken zu erkennnen. Beim Erkennen einer Ecke ist der PID-Regler zu verlassen, um die Ecke herumzufahren, den Roboter ernuet parallel auszurichten und dann wieder in den PID-Reglers zu gehen.

```
angleOld = get_angle(pos_o)
distOld = angles_dists_thread.threadOutput.dist
print(distOld)
time.sleep(0.07)
angleNew = get_angle(pos_o)
distNew = angles_dists_thread.threadOutput.dist
print(distNew)
if distNew > 0:
  if abs(distOld-distNew) >= threshold:                    ###here it checks for
the jump and if it finds one it will save its location
      time.sleep(0.1)
      distNew = angles_dists_thread.threadOutput.dist
      tankDrive.on(SpeedPercent(0), SpeedPercent(0))
      rightCornerAngle = angleNew
      rightCornerDist = distNew + 10
      print(rightCornerAngle)#debug
      print(rightCornerDist)#debug
      break
```

detect_houses_advanced.py

Drehen

Bei der gyrosensor.angle Funktion gab es das Problem, dass die Kalibrierung und Reset() Funktion nicht richtich funktioniert hat. Darum musste die circle_angle() Funktion verwendet werden. Dies führt zu weiteren Problem bei Überschreitung der 360° Grenze. Dies haben wir durch Fallanalysen in den Drehfunktionen gelöst.

turning.py namentlich: turn_right_to_x, turn_left_to_x, turn_to_x_left, turn_to_x_right, relative_turnv

Aufgabenstellung

Die Aufgabenstellung erfolgte durch den Fachbereich des Studiengangs technische Kybernetik der Universität Stuttgart.

Die Aufgabenstellung

Durch Industrie 4.0, individuell gefertigte Produkte und zunehmendes Onlineshopping steigt die Zahl der zu verteilenden Güter stark an. Während Industrie 4.0 und Produktion on-demand vielversprechende Möglichkeiten sind, Ressourcen effizient zu nutzen, stellen diese Entwicklungen neue Herausforderungen an die Logistik. Um diesen Herausforderungen zu begegnen und gleichzeitig umweltfreundliche Lieferketten aufzubauen, arbeiten Paketlieferdienste an autonomen Lieferfahrzeugen. Solch ein Fahrzeug soll im diesjährigen Roborace-Wettbewerb konstruiert werden. Die Aufgabe des Roboters ist es, mehrere Häuser anzufahren und dort Pakete abzuliefern. (weitere Details siehe [1])

Literatur

[1] Aufgabenstellung Roborace 2024: in elektronischer Form als lokale Datei Aufgabenstellung_Roborace2024.pdf und auf der Internetseite der Universität Stuttgart https://www.ist.uni-stuttgart.de/de/lehre/roborace/aufgabe/Aufgabenstellung_Roborace2024.pdf

[2] Was ist ein PID-Regler?: https://www.ist.uni-stuttgart.de/de/lehre/roborace/informationen/

Danksagungen

Wir möchten folgenden Projekten danken, welche eine große Hilfe für uns waren.

Betriebssystem und Bibliothek

Betriebssystem und Bibliothek, welche die Grundlage für unser Projekt darstellen. https://www.ev3dev.org/docs/getting-started/

Dokumentation

Die Bibliotheksdokumentation. https://ev3dev-lang.readthedocs.io/projects/python-ev3dev/en/stable/index.html

Simulation

Simulationen, um das Programm zu testen, ohne den ev3 physisch nutzen zu können.

3D Simulation 2D Simulation

> Konfigurationsdateien können im Repository unter miscellaneous gefunden werden.

Namensgebungskonventionen

Namensgebungskonventionen für sowohl Commits als auch den Quellcode, um das Projekt lesbar zu halten.

Commitkonventionen:
https://gist.github.com/qoomon/5dfcdf8eec66a051ecd85625518cfd13

Programmierkonventionen: https://android.googlesource.com/platform/external/google-styleguide/+/refs/tags/android-s-beta-2/pyguide.md